

### CS422 Computer Networks (Fall 2025): Homework 3

1. (5 points) TCP waits until it has received three duplicate ACKs before performing a fast retransmit. Why do you think the TCP designers chose not to perform a fast retransmit after the first duplicate ACK for a segment is received?

Most of the TCP designers chose to wait for three duplicate ACKs because a single duplicate ACK might result from packet reordering in the network, where segments arrive out of order but none are lost. One single dup ACK doesn't reliably indicate loss, but three dups suggest that subsequent packets have arrived, implying the missing segment was likely lost. This avoids unnecessary retransmissions and don't let loss time that could waste bandwidth.

2. (5 points) Why is a 2-way handshake not sufficient for reliable TCP connection establishment, and why do we need a 3-way handshake instead?

A 2-way handshake is insufficient because it could allow old, duplicated SYN packets from previous connections to erroneously establish a new connection on one side (the server might accept a stale SYN and send SYN-ACK, but the client didn't initiate it). The 3-way handshake ensures synchronization and agreement from both sides: the client's SYN, server's SYN-ACK (acknowledging the client's SYN), and client's ACK (acknowledging the server's SYN). This prevents half-open connections from delayed packets.

3. (6 points) After one party sends the final ACK in response to the peer's FIN segment during TCP connection termination: Why must this host remain in the waiting state for a certain period before fully closing the connection? If another FIN segment arrives before the timer expires, what will happen, and how does the host handle it?

After sending the final ACK responding to the peer's FIN, the host enters the TIME\_WAIT state for  $2 * MSL$ . This wait make that if the final ACK is lost, the peer can retransmit its FIN, and the host can resend the ACK to properly close the connection. It also allows any delayed segments from this connection to expire, preventing them from interfering with future connections using the same ports. If another FIN arrives before the timer expires, this makes the host resends the ACK and resets the TIME\_WAIT timer.

4. (6 points) What is the difference between flow control and congestion control in TCP? Assume the CWND is 10 MSS, while the RWND is 8 MSS, what will be the effective transmission window size?

Flow control in TCP prevents the sender from overwhelming the receiver's buffer capacity, using the receiver's advertised window (RWND) to limit how much data can be sent. The congestion control prevents the sender from overwhelming the network in general, adjusting the congestion window (CWND) based on perceived network congestion (e.g., via packet loss). The effective

transmission window size is the minimum of CWND and RWND, so  $\min(10 \text{ MSS}, 8 \text{ MSS}) = 8 \text{ MSS}$ .

5. (6 points) TCP congestion control uses AIMD for fairness. Please think about what will happen if TCP CC uses AIAD (additive increase and additive decrease (increase/decrease by one)) or MIAD (multiplicative increase and additive decrease). Please follow the diagram plotted in the lecture to see what will happen with two flows.

TCP's AIMD first gradually increases CWND and then halves it on congestion. Then according to AIAD, flows would maintain their relative shares without converging to fairness because the increase and decrease vectors would be parallel. Unfair bandwidth allocation would result from a flow that starts with a larger CWND staying ahead. However, when using MIAD, the additive decrease reduces slowly, while the multiplicative increase would result in aggressive growth that could overshoot capacity. Flows may oscillate or deviate from the efficiency line in the phase plot, which could cause instability, unfairness, or frequent congestion collapses without properly convergent congestion.

6. (12 points) Assume TCP congestion control uses all the following algorithms covered in our lecture: slow start (SS), congestion avoidance (CA), fast retransmit and fast recovery, and retransmission upon timeout.

All the following assumptions are the same as those used in class unless specified (say, cwnd, ssthresh and which packet gets lost).

- MSS = 1B. Initially ssthresh at the sender is set to 4, and cwnd as 1. Assume cwnd and ssthresh are measured in segments (or in bytes as MSS = 1B). The transmission time for each segment is negligible.

Retransmission timeout (RTO) is initially set to 500 ms at the sender and is unchanged during the connection lifetime. RTT is 100ms for all transmissions.

- The connection starts to transmit data at time  $t = 0$ , and the initial sequence number starts from 1. Segment with sequence number 9 is lost once. No other segments are lost.

- The receiver acknowledges every segment (no delayed ACK) and uses TCP cumulative ACK (i.e.,  $\text{ACK} = (\text{largest sequence received in order at the receiver} + 1)$ ). Besides, the sender always has data available for transmission.

- When ssthresh equals to cwnd, we use slow start, not congestion avoidance.

- Right after fast retransmit/fast recovery phase, if ssthresh equals to cwnd, use the slow start algorithm.

- The receiver will buffer out of order packets (like selective repeat).

When does the sender receive the ACK for the segment with the sequence number 15? Which algorithms will be used upon receiving ACK for the segments with the sequence number 15 and 16? Show your diagram. (hints: please follow the example diagram in class to show congestion control step-by-step.)

ACK 16 (ACK 16 cumulatively acknowledges up to seq 15) is sent to the sender at  $t = 500$  ms for the segment with sequence number 15. The rapid recovery method is used to end the fast recovery phase and set  $CWND$  to  $ssthresh$  (deflating it) after getting this ACK 16. Since  $CWND == ssthresh$  after exiting recovery, the delayed start procedure is applied after getting the ACK for segment 16 (ACK 17).

